

Mini Project Sistem Basis Data

TradeSpace[●]

Smart Tech Marketplace for University Students

Group PAGI 2

1. Muhammad Dhiya'ulhaq (2406356782)
2. Irgy Rabbani Sakti (2406438290)
3. Dimar Ilham Tamara (2406413804)
4. Valiant Joshua (2406352153)
5. Ade Zaskia Azzahra (2406344353)

The Real-World Case Study

eBay

eBay merupakan salah satu marketplace terbesar di dunia yang menyediakan platform jual beli berbagai jenis produk secara online.

eBay menangani jutaan produk dari berbagai kategori, seperti laptop, smartphone, fashion hingga elektronik, dimana setiap produk memiliki spesifikasi dan atribut yang berbeda beda

Alasan menggunakan MongoDB / Document Store:

1. Mendukung struktur data yang fleksibel dan dinamis untuk berbagai jenis produk
2. Setiap produk dapat memiliki atribut yang berbeda tanpa harus membuat banyak tabel
3. Mengurangi penggunaan operasi JOIN yang kompleks
4. Lebih scalable untuk menangani jutaan data marketplace
5. Memungkinkan embedding data dalam satu dokumen

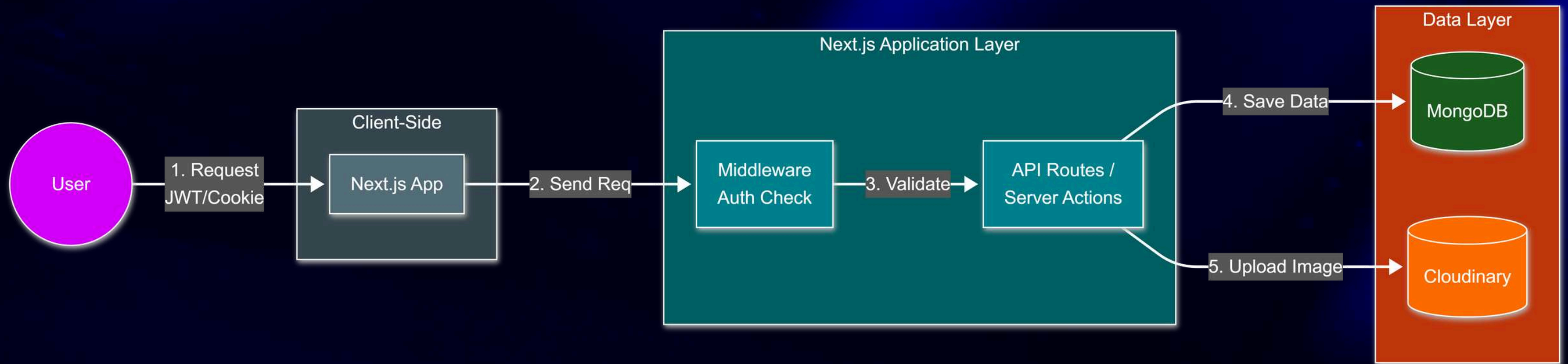
Problem

- Mahasiswa masih kesulitan menemukan marketplace gadget yang khusus dan terpercaya untuk lingkungan kampus.
- Transaksi jual beli masih banyak dilakukan melalui grup chat atau media sosial yang tidak terstruktur dan sulit digunakan untuk mencari produk tertentu.
- Marketplace umum belum mendukung spesifikasi elektronik yang fleksibel dan beragam, seperti RAM, CPU, storage, GPU, dan baterai.
- Tidak adanya verifikasi mahasiswa meningkatkan risiko penipuan dan membuat transaksi menjadi kurang aman.
- Database relasional tradisional membutuhkan banyak tabel dan operasi JOIN yang dapat menurunkan performa sistem marketplace berskala besar.

Motivation

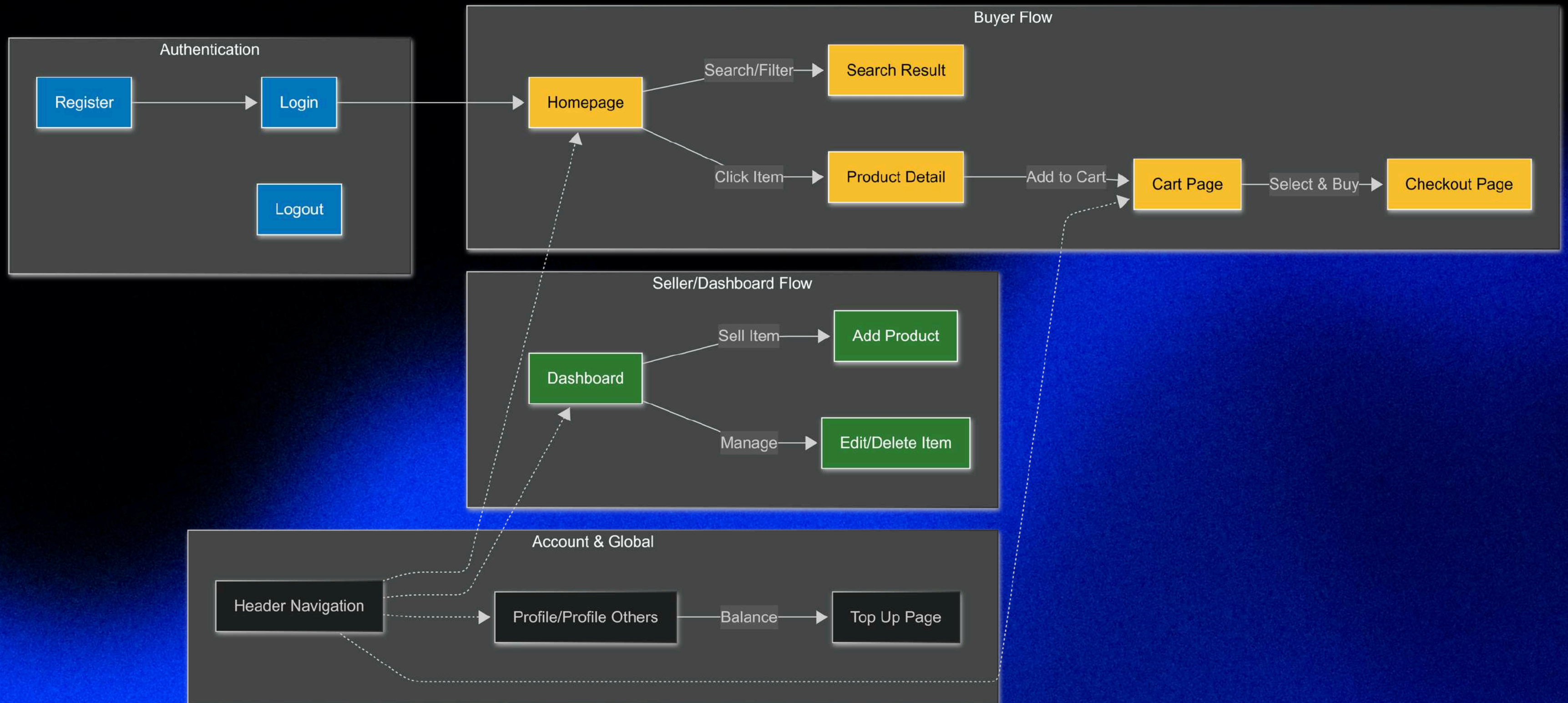
- Perkembangan teknologi membuat mahasiswa semakin bergantung pada perangkat elektronik.
- Mahasiswa sering melakukan jual beli atau upgrade perangkat teknologi, namun masih kesulitan menemukan platform yang aman.
- Produk elektronik memiliki spesifikasi yang sangat beragam dan terus berkembang.
- MongoDB sebagai Document Store database mampu menyimpan data produk dengan struktur yang fleksibel.
- TradeSpace dikembangkan untuk menciptakan ekosistem marketplace teknologi yang lebih terstruktur, aman, dan mudah digunakan oleh mahasiswa

Design System



- Frontend & Backend: Next.js
- Autentikasi: JWT & HTTP-Only Cookie
- Database: MongoDB Atlas
- Penyimpanan: Cloudinary
- Keamanan: Middleware

User Flow



Data Modeling: User Collection

```
const UserSchema = new mongoose.Schema({
  username: {
    type: String, required: true
  },
  email: {
    type: String, required: true, unique: true
  },
  password: {
    type: String, required: true,
  },
  is_verified: {
    type: Boolean, default: false,
  },
  balance: {
    type: Number, default: 0,
  },
  cart: {
    type: [CartSchema]
  }
}, {timestamps: true});
```

```
const CartSchema = new mongoose.Schema({
  item_id: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'Item',
    required: true
  },
  qty: {
    type: Number,
    required: true
  },
  price_snap: {
    type: Number,
    required: true
  }
});
```

- Embedding Strategy: Menggunakan pola denormalization dengan embedding data Cart langsung ke dalam dokumen User.
- Performance Optimization: Mengurangi database round trip dengan menggabungkan profil user dan isi cart dalam satu operasi query.

Data Modeling: Item Collection

```
const ItemSchema = new mongoose.Schema({
  seller_id: {
    type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true
  },
  name: {
    type: String, required: true
  },
  description: {
    type: String, required: true
  },
  picture_url: {
    type: String, required: true
  },
  category: {
    type: String, required: true
  },
  specs: {
    type: mongoose.Schema.Types.Mixed
  },
  price: {
    type: Number, required: true
  },
  stock: {
    type: Number, required: true, min: 0
  },
  condition: {
    type: String, required: true
  },
  average_rating: {
    type: Number, default: 0
  },
  reviews: [ReviewSchema]
}, {timestamps: true});
```

```
export const ReviewSchema = new mongoose.Schema({
  from_user: {
    type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true
  },
  rating: {
    type: Number, required: true, min: 1, max: 5
  },
  comment: {
    type: String
  },
  timestamp: {
    type: Date, default: Date.now
  }
}, {_id: true});
```

- Embedding Strategy: Menggunakan pola denormalization dengan embedding data Review langsung ke dalam dokumen Item.
- Performance Optimization: Mengurangi database round trip dengan menggabungkan data item dan review dalam satu operasi query.

Data Modeling: Category Collection

```
const CategorySchema = new mongoose.Schema({
  name: {
    type: String,
    required: true,
    unique: true
  },

  allowed_specs: [{
    type: String,
    required: true
  }]
});
```

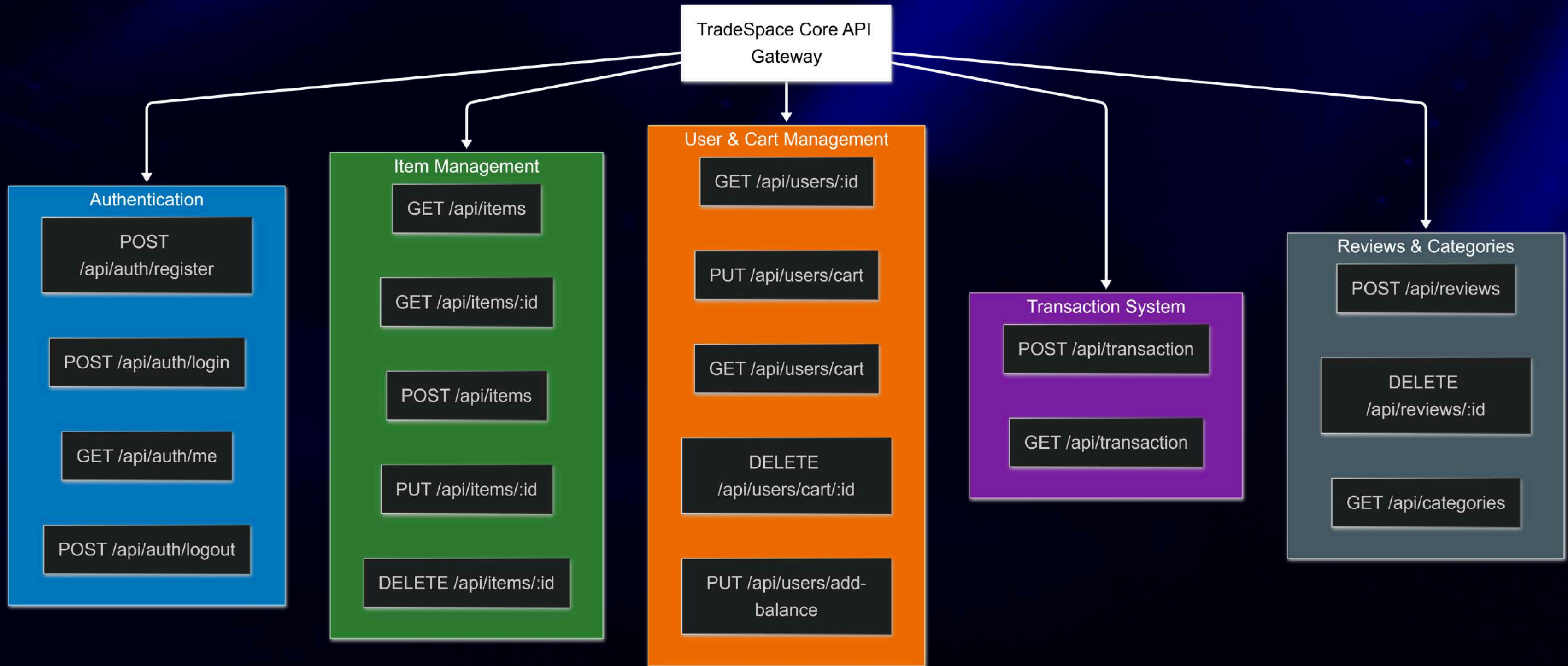
- Merupakan metadata standar kategori untuk item.
- Mempermudah filtering data berdasarkan kategori.

Data Modeling: Transaction Collection

```
const TransactionSchema = new mongoose.Schema({
  buyer_id: {
    type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true
  },
  seller_id: {
    type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true
  },
  item_snapshot: {
    name: {
      type: String,
      required: true
    },
    price_paid: {
      type: Number,
      required: true
    }
  },
  status: {
    type: String,
    enum: ['pending', 'completed', 'cancelled'],
    default: 'completed'
  },
  total_amount: {
    type: Number,
    required: true
  }
}, {timestamps: true})
```

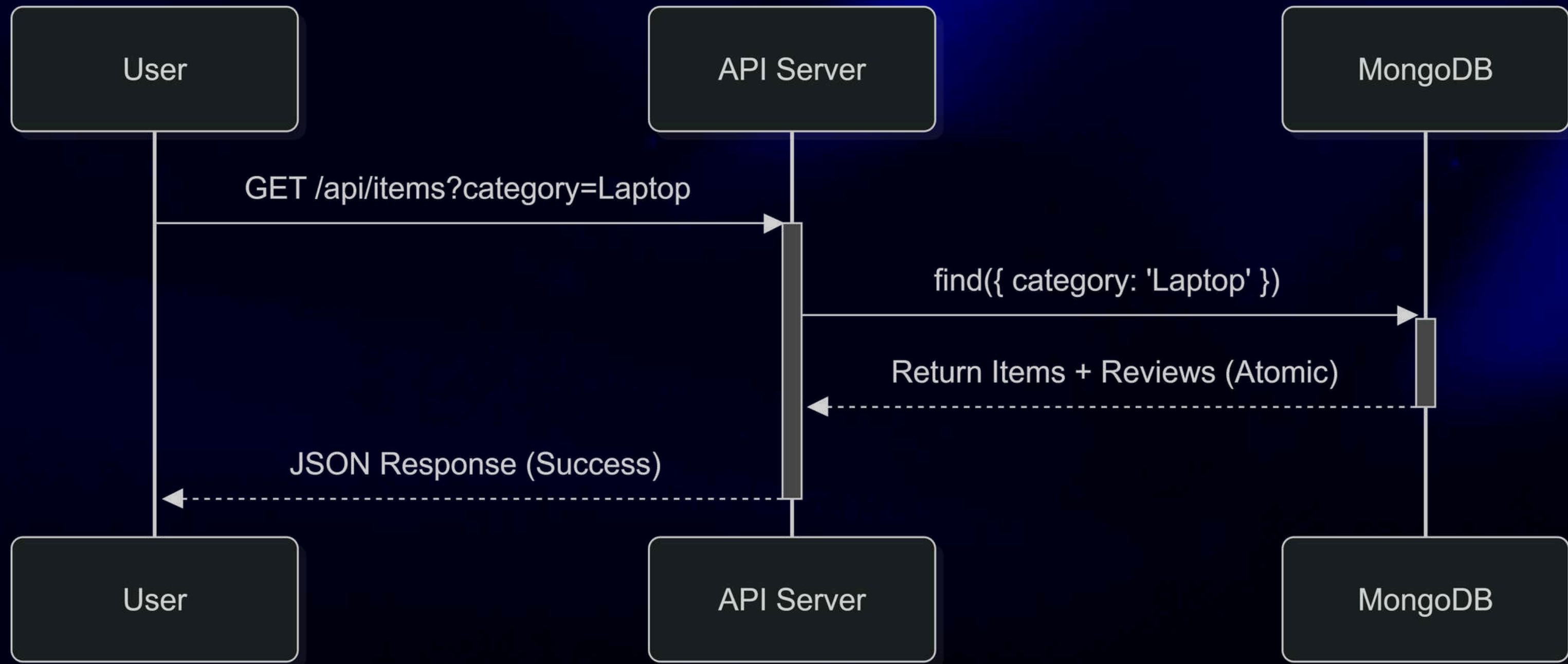
- Immutable Historical Record
- Referential Integrity
- Transaction Lifecycle
- Audit Trail

API



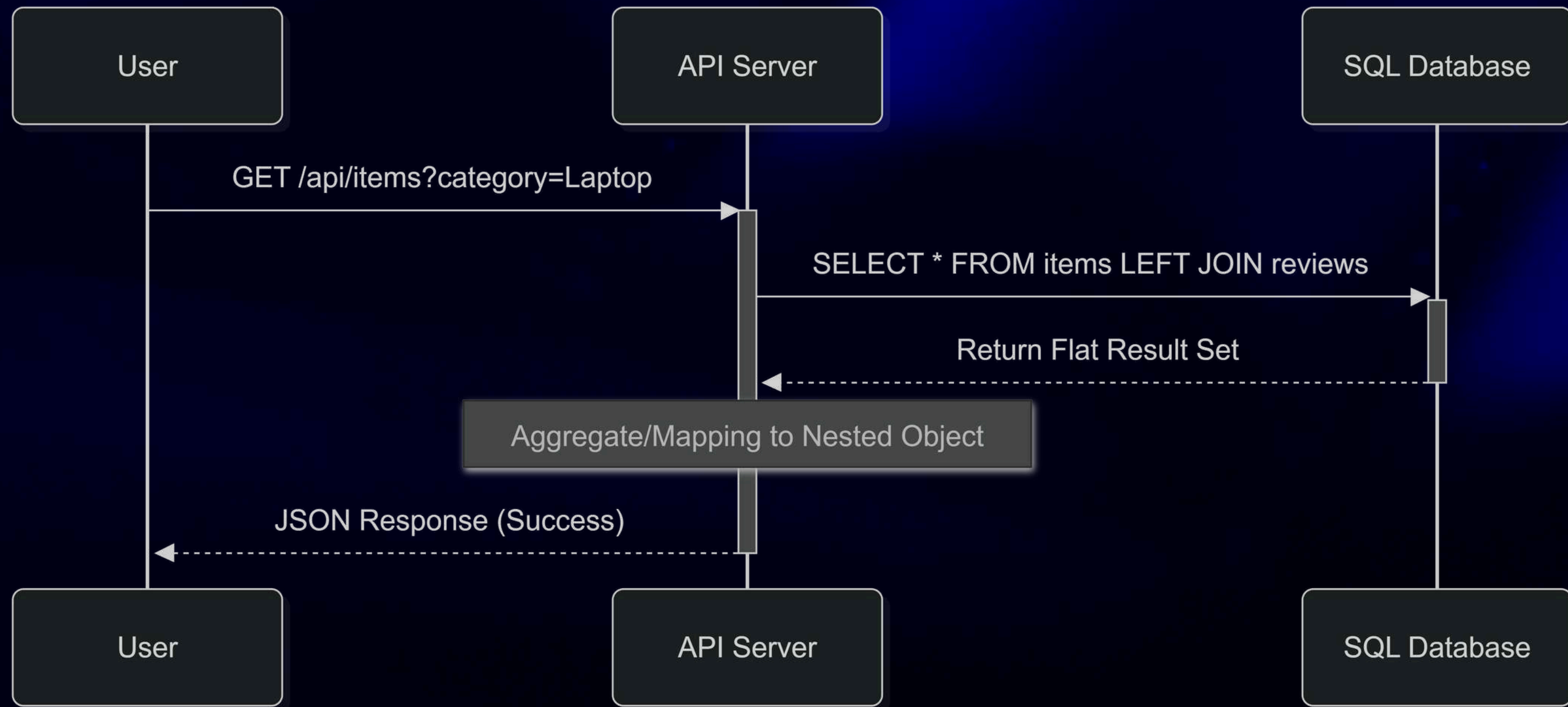
Perbandingan Read Operation NoSQL dan SQL

UML API GET /api/items (MongoDB)



Perbandingan Read Operation NoSQL dan SQL

UML API GET /api/sql/items (PostgreSQL)



Perbandingan Read Operation NoSQL dan SQL

Stress Testing API GET /api/items (MongoDB)

```
C:\WebDev\TradeSpace\trade-space>k6 run --summary-export=nosql-results.json load-tests/get-items-nosql.js
```



```
execution: local
  script: load-tests/get-items-nosql.js
  output: -
```

```
scenarios: (100.00%) 1 scenario, 20 max VUs, 1m20s max duration (incl. graceful stop):
  * default: Up to 20 looping VUs for 50s over 3 stages (gracefulRampDown: 30s, gracefulStop: 30s)
```

TOTAL RESULTS

```
checks_total.....: 5065    101.178165/s
checks_succeeded...: 100.00% 5065 out of 5065
checks_failed.....: 0.00%   0 out of 5065
```

✓ status is 200

HTTP

```
http_req_duration.....: avg=159.26ms min=46.29ms med=83.79ms max=746.11ms p(90)=588.73ms p(95)=637.8ms
{ expected_response:true }...: avg=159.26ms min=46.29ms med=83.79ms max=746.11ms p(90)=588.73ms p(95)=637.8ms
http_req_failed.....: 0.00%   0 out of 5065
http_reqs.....: 5065    101.178165/s
```

EXECUTION

```
iteration_duration.....: avg=159.27ms min=46.29ms med=83.79ms max=746.11ms p(90)=588.73ms p(95)=637.8ms
iterations.....: 5065    101.178165/s
vus.....: 1    min=1    max=20
vus_max.....: 20    min=20    max=20
```

NETWORK

```
data_received.....: 50 MB  988 kB/s
data_sent.....: 547 kB  11 kB/s
```

```
running (0m50.1s), 00/20 VUs, 5065 complete and 0 interrupted iterations
default ✓ [=====] 00/20 VUs  50s
```

Perbandingan Read Operation NoSQL dan SQL

Stress Testing API GET /api/sql/items (PostgreSQL)

```
C:\WebDev\TradeSpace\trade-space>k6 run --summary-export=sql-results.json load-tests/get-items-sql.js
```



```
execution: local
script: load-tests/get-items-sql.js
output: -
```

```
scenarios: (100.00%) 1 scenario, 20 max VUs, 1m20s max duration (incl. graceful stop):
* default: Up to 20 looping VUs for 50s over 3 stages (gracefulRampDown: 30s, gracefulStop: 30s)
```

TOTAL RESULTS

```
checks_total.....: 1293    25.846328/s
checks_succeeded...: 100.00% 1293 out of 1293
checks_failed.....: 0.00%   0 out of 1293
```

✓ status is 200

HTTP

```
http_req_duration.....: avg=625.97ms min=287.17ms med=613.41ms max=2.69s p(90)=697.37ms p(95)=854.58ms
{ expected_response:true }...: avg=625.97ms min=287.17ms med=613.41ms max=2.69s p(90)=697.37ms p(95)=854.58ms
http_req_failed.....: 0.00%   0 out of 1293
http_reqs.....: 1293    25.846328/s
```

EXECUTION

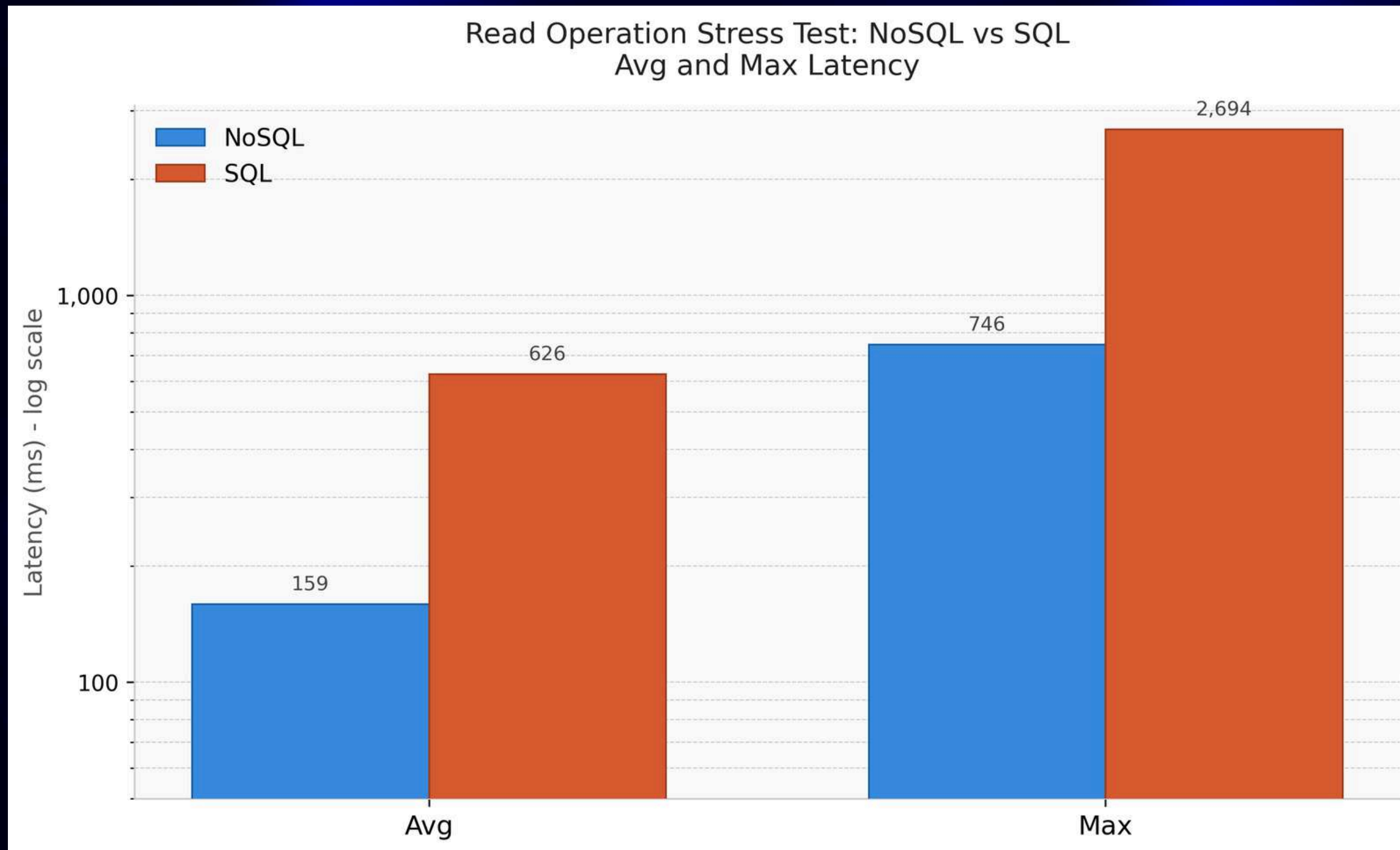
```
iteration_duration.....: avg=626.07ms min=287.17ms med=613.58ms max=2.69s p(90)=697.51ms p(95)=854.58ms
iterations.....: 1293    25.846328/s
vus.....: 1    min=1    max=20
vus_max.....: 20    min=20    max=20
```

NETWORK

```
data_received.....: 6.9 MB 139 kB/s
data_sent.....: 145 kB 2.9 kB/s
```

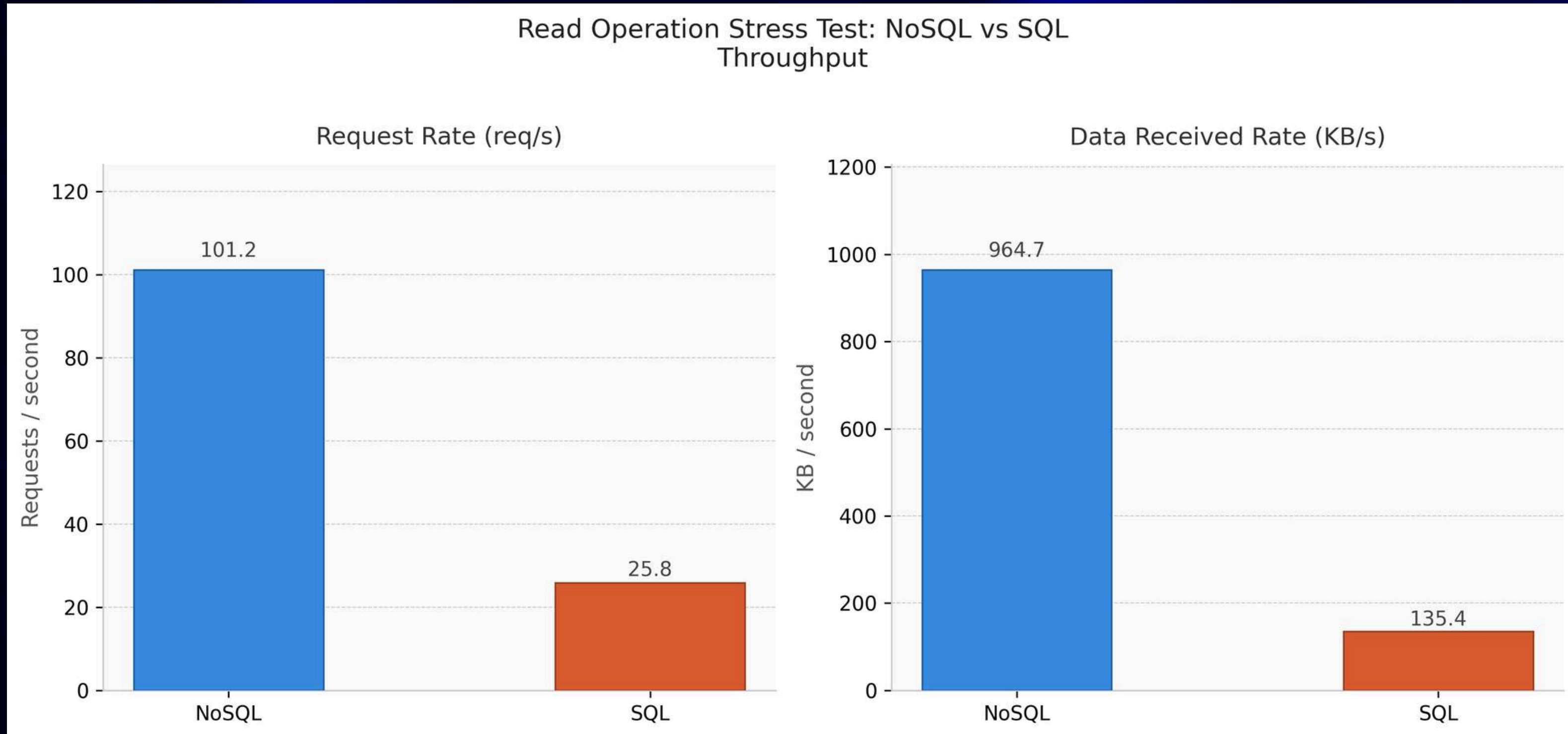
```
running (0m50.0s), 00/20 VUs, 1293 complete and 0 interrupted iterations
default ✓ [=====] 00/20 VUs 50s
```

Grafik Perbandingan Stress Testing MongoDB dan PostgreSQL



- MongoDB 4x lebih cepat pada avg latency: 159ms vs 626ms
- MongoDB 3.6x lebih cepat pada max latency: 746ms vs 2.694ms

Grafik Perbandingan Stress Testing MongoDB dan PostgreSQL



- MongoDB menangani 4x lebih banyak request: 101.2 req/s vs 25.8 req/s
- Data throughput MongoDB 7x lebih tinggi: 964.7 KB/s vs 135.4 KB/s

Lessons Learned & Architectural Trade-offs

- Performance vs. Consistency: Pemilihan NoSQL (MongoDB) memprioritaskan read-throughput dan latensi, dengan konsekuensi pada hilangnya rigiditas ACID transactions yang biasanya menjadi standar pada RDBMS.
- Denormalization vs. Normalization: Strategi embedding mempercepat proses read-heavy, tetapi menuntut manajemen integritas data yang lebih hati-hati di sisi aplikasi.

Individual Contribution

- Dimar Ilham Tamara | Backend Developer
 - Mendesain arsitektur web
 - Merancang dan membuat struktur database NoSQL
 - Membuat RESTful API untuk CRUD users, items, reviews, dan categories
 - Membuat user flow
- Irgy Rabbani Sakti | Frontend Developer
 - Penerapan sistem "Cart" untuk mentotalkan item yang dibeli
 - Mengembangkan product page yang mampu melakukan fetching data spesifikasi, stok, dan kategori secara real-time.
 - Menerapkan sistem review pembeli, termasuk fitur review dengan stars, komentar, dan tanggal real-time

Individual Contribution

- Ade Zaskia Azzahra | Frontend Developer
 - Membuat tampilan homepage, login page, dan register page.
 - Mengimplementasikan navigasi dan komponen Navbar pada website
 - Menghubungkan frontend dengan backend menggunakan API fetching untuk menampilkan data produk secara dinamis
 - Mengimplementasikan sistem autentikasi frontend untuk fitur login dan register
 - Melakukan perbaikan UI dan responsive layout agar tampilan lebih konsisten dan user-friendly
- Valiant Joshua | Backend Developer
 -

Individual Contribution

- Muhammad Dhiya'ulhaq |
 - Membuat halaman frontend. Search, Dashboard, Sell, Checkout, Top Up, Transactions, dan User Profile.
 - Fitur belanja lengkap. Browse produk, cart dengan checkbox select, hingga checkout.
 - Fitur seller. Form jual produk dengan drag-and-drop upload gambar, edit listing, dan dashboard statistik penjualan.
 - Detail UX. Search dengan filter, sort, pagination; modal profil seller; format harga ribuan; URL sync saat filter.

Thank You

Github: <https://github.com/Dimar-I-T/trade-space/tree/main>

Website: <https://trade-space-eight.vercel.app>